

What We Noticed

- The skill was calibrated mainly on backend repos and has almost no frontend architectural knowledge yet (only a partial React file), so it tends to score unfamiliar frontend patterns lower than it should.
- Because of this, even gold-standard public repos and clean internal reference repos scored below 100 (low 90s) when we tested them, even though there were no real gaps to find.
- Some scoring rules seem built for backend or open-source projects and don't fit web projects well. For example: it treats a full TypeScript migration as mandatory, when in practice plain JavaScript is still a valid, first-class choice — React itself was built by Facebook/Meta with JSX for plain JavaScript, and Meta's own type-checking tool for React (Flow) was designed so teams could keep writing .js/.jsx files instead of requiring TypeScript. Many widely-used libraries, like Moment.js, are written entirely in plain JavaScript too. Going forward, we can think about TS, but a deduction in score on this basis is not right.
- It penalizes non-latest framework/language versions based on recency alone, without actually scanning for vulnerabilities to justify the deduction.
- It applies open-source hygiene standards (LICENSE, CONTRIBUTING, SECURITY, CHANGELOG, branch protection) to internal Azure repos, where branch/merge protections actually live in Azure DevOps settings that the skill can't see.
- It doesn't yet detect frontend-specific code quality issues (accessibility, SEO, direct DOM access vs. framework APIs) — it relies on regex only, with no real parsing.
- We also found a small lockfile-detection bug: it flags "multiple lockfiles" by mistake, because it's matching lockfiles that belong to dependencies, not the project itself.

What We've Already Done

We took a local copy of the skill to understand how it works, and started drafting some fixes.

- Added real frontend architecture knowledge: new nextjs.md, angular.md, rewritten react.md, plus a component-based-architecture reference.
- Updated JS/TS/Node/ECMAScript version knowledge.
- Reworked version-recency scoring: older-but-supported versions (React 18, Next.js 14, Angular 18–20, older-but-supported Python/Go) are no longer penalized — only genuinely EOL versions (AngularJS 1.x, Node ≤ 20 , Python 2) incur deductions; upgrade suggestions are now notes, not score hits.
- Improved the platform-detection script to correctly identify component-based architecture and prioritize Next.js detection over generic React.

- Updated the rubric to default repos to "internal" unless an OSS license is present — this waives LICENSE/CONTRIBUTING/SECURITY/CHANGELOG and branch-protection expectations for internal repos.
- Added accessibility/SEO/code-quality checks, plus a supplementary Node script for deeper analysis (e.g., function complexity) beyond regex.
- Added a hard rule that every point deduction from 100 must be tied to an explicitly listed gap in the report — no silent/unexplained deductions.
- Fixed the lockfile-detection bug causing false "multiple lockfiles" positives.